



POO - Unid III

Exercício 1:

1- Um sistema decide usar `Dictionary<Produto, decimal>` para mapear um produto para um desconto. O objetivo é que duas instâncias distintas de `Produto`, criadas separadamente, sejam tratadas como a mesma chave quando tiverem o mesmo `Id`. Sem fornecer um `IEqualityComparer<Produto>` no construtor do dicionário, qual conjunto de mudanças é necessário para que esse comportamento passe a ser possível de forma consistente?

A)

Implementar `IComparable<Produto>` e, em `CompareTo`, ordenar pelo `Id`, pois o dicionário usa ordenação para localizar chaves.

B)

Sobrescrever `ToString()` para retornar o `Id`, pois o dicionário compara chaves pelo texto que elas exibem.

C)

Implementar `IEquatable<Produto>` e sobrescrever `Equals(object)` e `GetHashCode()` de modo coerente com o `Id`, pois o dicionário depende de igualdade e de código de hash compatíveis.

D)

Tornar `Id` um campo public mutável, porque o dicionário precisa atualizar o valor do `Id` para resolver colisões.

E)

Marcar a classe `Produto` como `sealed`, porque o dicionário só aceita como chave tipos que não permitem herança.

Exercício 2:

Considere a diferença entre coleções genéricas e coleções antigas que armazenam elementos como object. No trecho abaixo, qual descrição das operações e do risco geral associado a esse padrão de uso do ArrayList está correta?

```
System.Collections.ArrayList lista = new System.Collections.ArrayList();  
lista.Add(5);  
int x = (int)lista[0];
```

A)

Ocorre unboxing na chamada Add(5) e ocorre boxing no cast (int)lista[0], e o risco principal é uma NullReferenceException.

B)

Ocorre boxing na chamada Add(5) e ocorre unboxing no cast (int)lista[0]; se o elemento recuperado não contiver um int empacotado, o cast pode falhar em tempo de execução.

C)

Não ocorre boxing nem unboxing, porque o compilador reconhece que o valor original era um literal inteiro e elimina a conversão.

D)

Ocorre boxing em Add(5) e não ocorre unboxing no cast, porque o runtime faz conversões numéricas implícitas ao recuperar valores de object.

E)

Ocorre boxing em Add(5); a atribuição object o = lista[0]; não envolve boxing/unboxing; e o cast (int)o é sempre seguro porque o compilador impede que outros tipos sejam inseridos em ArrayList.

Exercício 3:

Considere o método genérico a seguir, que precisa (i) instanciar T com `new T()`, (ii) ler e escrever a propriedade `Id` em uma variável do tipo T, e (iii) comparar duas instâncias de T por meio de `CompareTo` sem casts. Qual cláusula `where` é simultaneamente bem formada em C# e suficiente para que o corpo do método compile?

A)

`where T : new(), EntidadeBase, IComparable<T>`

B)

`where T : EntidadeBase, new(), IComparable<T>`

C)

`where T : EntidadeBase, IComparable<T>, new()`

D)

`where T : class, EntidadeBase, IComparable<T>, new()`

E)

`where T : EntidadeBase, IComparable<T>`

Exercício 4:

Em um cenário no qual existe uma classe `Cliente` que implementa `IEquatable<Cliente>` (com `Equals(Cliente other)`), sobrescreve `Equals(object)` e `GetHashCode()`, e também existe um comparador externo `ClientePorNomeIgualdade : IEqualityComparer<Cliente>` (que ignora maiúsculas/minúsculas no nome e calcula hash coerente com esse critério), analise as duas operações: (i) inserir elementos em um `HashSet<Cliente>` construído com `new ClientePorNomeIgualdade()`, e (ii) executar `clientes.Contains(procurado)` em uma `List<Cliente>` para um `procurado` com os mesmos dados de um elemento já existente. Qual alternativa descreve corretamente quais implementações de igualdade são consultadas em cada operação?

A)

No HashSet, a unicidade é decidida por `Cliente.Equals(object)` e `Cliente.GetHashCode()`, enquanto em `List.Contains` a verificação é feita por `ClientePorNomeIgualdade.Equals`.

B)

No HashSet, a unicidade é decidida por `ClientePorNomeIgualdade.Equals` e `ClientePorNomeIgualdade.GetHashCode`, enquanto em `List.Contains` a verificação é feita via `EqualityComparer<Cliente>.Default`, que usa `IEquatable<Cliente>` quando disponível.

C)

No HashSet, a unicidade é decidida por `IComparable<Cliente>.CompareTo`, enquanto em `List.Contains` a verificação é feita por `Cliente.CompareTo`.

D)

No HashSet, primeiro se chama `Cliente.Equals(Cliente other)` e depois `ClientePorNomeIgualdade.GetHashCode`, enquanto em `List.Contains` se compara por identidade de referência e não por igualdade lógica.

E)

No HashSet, a unicidade é decidida por `ClientePorIdComparer.Compare`, enquanto em `List.Contains` a verificação só usa `IEquatable<Cliente>` se a lista estiver previamente ordenada.

Exercício 5:

Suponha que uma lista contenha exatamente estes quatro elementos, todos do tipo `Cliente`:

(Id=2, Nome="Maria")
(Id=1, Nome="João")
(Id=3, Nome="Ana")
(Id=4, Nome="Gerado com new()")

Em seguida, a lista é ordenada por Id usando um `IComparer<Cliente>` externo e, logo depois, é chamada a função genérica `EncontrarMaximo<T>(IEnumerable<T> itens)` where `T : IComparable<T>`, que percorre a sequência e escolhe o maior elemento segundo `CompareTo`. Considerando que `Cliente.CompareTo` compara Nome usando `StringComparison.Ordinal`, qual elemento será retornado por `EncontrarMaximo`?

A)

Será retornado (Id=4, Nome="Gerado com new()"), pois a ordenação por Id determina o "maior" elemento usado pelo algoritmo.

B)

Será retornado (Id=2, Nome="Maria"), pois o algoritmo decide pelo resultado de CompareTo e, nesse conjunto, o nome iniciado por "M" vence as alternativas iniciadas por "A", "G" e "J" na comparação ordinal.

C)

Será retornado (Id=1, Nome="João"), pois caracteres acentuados tornam o nome ordinalmente maior do que os demais.

D)

Será retornado (Id=3, Nome="Ana"), pois o primeiro elemento após a ordenação por Id é tratado como máximo final.

E)

Será lançada InvalidOperationException, pois, após a ordenação com IComparer externo, o método não consegue estabelecer um máximo sem uma ordenação natural prévia.

Exercício 6:

Em uma consulta escrita com a sintaxe from ... where ... select ... cujo alvo implementa IQueryable<T>, qual alternativa descreve com mais precisão o que o compilador gera e por que isso permite que um provedor (por exemplo, um ORM) execute a consulta fora do CLR?

A)

O compilador executa a filtragem durante a compilação, reduzindo a sequência a um conjunto constante, e o provedor apenas materializa esse conjunto.

B)

O compilador traduz a consulta para chamadas encadeadas de operadores-padrão, e as lambdas envolvidas podem ser representadas como árvores de expressão, permitindo que o provedor inspecione a estrutura da consulta.

C)

O compilador traduz a consulta para chamadas de Enumerable e converte as lambdas em delegates comuns, o que obriga o provedor a receber um `Func<T,bool>` já "pronto" e a executar a filtragem em memória.

D)

O compilador preserva as palavras-chave (`from`, `where`, `select`) como "strings de consulta", que são repassadas ao provedor para serem interpretadas em tempo de execução.

E)

O compilador substitui a consulta por uma chamada única a `Aggregate`, pois toda consulta pode ser reduzida a um acumulador, e o provedor executa esse acumulador diretamente no banco.

Exercício 7:

Considere o programa a seguir. Sem executar, qual alternativa fornece os valores corretos de `a` e `b` ao final da execução?

```
using System;
using System.Collections.Generic;
using System.Linq;

internal static class Program
{
    private static void Main()
    {
        var lista = new List<int> { 1 };

        IEnumerable<int> query = lista.Select(n => n * 10);

        int a = query.Count();

        lista.Add(2);

        int b = query.Count();

        Console.WriteLine($"{a}|{b}");
    }
}
```

```
}
```

A)

a = 1 e b = 1, porque Select materializa a sequência no momento da construção da consulta.

B)

a = 1 e b = 2, porque Count() força a enumeração da sequência naquele instante, e a consulta volta a percorrer a fonte quando chamada novamente.

C)

a = 2 e b = 2, porque Count() contabiliza também os elementos que serão adicionados depois, já que a consulta é "planejada" previamente.

D)

a = 10 e b = 20, porque Count() aplica a projeção e devolve o valor do último elemento projetado.

E)

O programa lança exceção, porque a fonte foi modificada após a criação de query, e o LINQ proíbe esse padrão.

Exercício 8:

Sobre GroupBy e Any em LINQ to Objects (isto é, operando sobre sequências que implementam IEnumerable<T>), qual alternativa está correta?

A)

GroupBy altera a sequência de origem ao anexar metadados de chave em cada elemento, e Any lança exceção quando aplicado a uma sequência vazia.

B)

GroupBy produz uma coleção de agrupamentos que expõem uma chave via Key e são iteráveis como subcoleções; Any() sem predicado verifica se a sequência possui ao menos um elemento e retorna false quando ela está vazia.

C)

GroupBy retorna diretamente um Dictionary<TKey, List<TElement>> pronto, e Any é equivalente a Contains em todos os casos, pois ambos só testam igualdade.

D)

GroupBy só pode agrupar por tipos numéricos, pois chaves textuais impedem a execução adiada; Any sempre avalia toda a sequência para decidir.

E)

GroupBy garante que os grupos saem ordenados pela chave, e Any ignora o predicado, tratando qualquer função booleana como se fosse apenas um teste de não vazio.

Exercício 9:

Considere o código abaixo (o método RegraNegocio é um método C# comum, sem tradução para SQL):

```
static bool RegraNegocio(Cliente c) => c.LimiteCredito > 5000m && c.Bloqueado == false;
```

```
var ids = contexto.Clientes
    .Where(c => c.Pais == "BR")
    .AsEnumerable()
    .Where(c => RegraNegocio(c))
    .Select(c => c.Id)
    .ToList();
```

A)

O banco de dados executa os dois filtros e devolve somente os Id já projetados, pois AsEnumerable() não altera a forma de execução.

B)

O banco de dados executa apenas o primeiro Where, depois o cliente aplica RegraNegocio, e a projeção de Id ocorre no cliente.

C)

O cliente executa ambos os Where, mas o banco de dados executa a projeção de Id por reconhecer `Select(c => c.Id)`.

D)

O banco de dados executa o primeiro Where e a projeção de Id, mas o cliente executa apenas o segundo Where.

E)

O cliente força a materialização completa antes do primeiro Where, porque `AsEnumerable()` equivale a `ToList()`.

Exercício 10:

Analise o efeito do tipo estático no trecho abaixo:

```
IQueryable<Pedido> pedidos = contexto.Pedidos; // origem remota
IEnumerable<Pedido> seq = pedidos;           // mudança de tipo estático

var r = seq.Where(p => p.Total > 100m).ToList();
```

A)

O filtro `p.Total > 100m` é traduzido para SQL, pois o objeto real continua sendo `IQueryable<Pedido>`.

B)

O filtro é executado no cliente, e o banco de dados recebe uma consulta sem o predicado do Where.

C)

O código lança exceção em tempo de execução, pois `Enumerable.Where` não aceita uma origem que também seja `IQueryable`.

D)

O banco de dados executa o filtro, mas o cliente executa a iteração, já que `ToList()` sempre roda fora do servidor.

E)

O filtro é executado no cliente somente se houver uma chamada explícita a `AsEnumerable()`, o que não ocorre.